

ADVISER CONSULTATION REFERENCE

Ynclino Apartment Management System

Complete Study Guide — What, Where & How

Framework: ASP.NET Core 8.0 MVC | **Database:** SQLite + Entity Framework Core 8

Language: C# | **UI:** Bootstrap 5 (local) | **Auth:** Cookie-based

Namespace: YnclinoApartmentManagementSystem

crud branch (Full)

All 6 modules
Login + Roles
Port: 7251

crud(copy) branch

Units + Tenants only
No login required
Port: 7250

Table of Contents

1. Two Branches Explained	3
2. Technology Stack	3
3. Project Folder Structure	4
4. Database Design	5
— tblUsers, tblUnits, tblTenants, tblBillings, tblMaintenanceRequests, tblLostFoundItems, tblClaimRequests	
5. Authentication & Role-Based Access	7
6. Module: Units Management	8
7. Module: Tenants Management	9
8. Module: Billing & Payment	11
9. Module: Maintenance Requests	12
10. Module: Lost & Found	13
11. Module: User Management	14
12. Dashboard (Home)	15
13. MVC Pattern — How It Applies Here	15
14. Key Files & What They Do	16
15. How to Demo the System	17
16. Glossary of Technical Terms	18
17. Adviser Q & A	19

1. Two Branches Explained

The project has two Git branches that serve different purposes during development.

crud branch (Full System)

Item	Value
Port (HTTPS)	localhost:7251
Login required?	Yes — cookie auth
Default user	admin / Admin@123
Modules	All 6 + Dashboard
Roles	Admin, SemiAdmin, Tenant
DB tables	7 tables

crud(copy) branch (Demo/Stripped)

Item	Value
Port (HTTPS)	localhost:7250
Login required?	No — auto-login as Admin
Default user	auto (no login screen)
Modules	Units + Tenants only
Roles	Admin (auto)
DB tables	3 tables

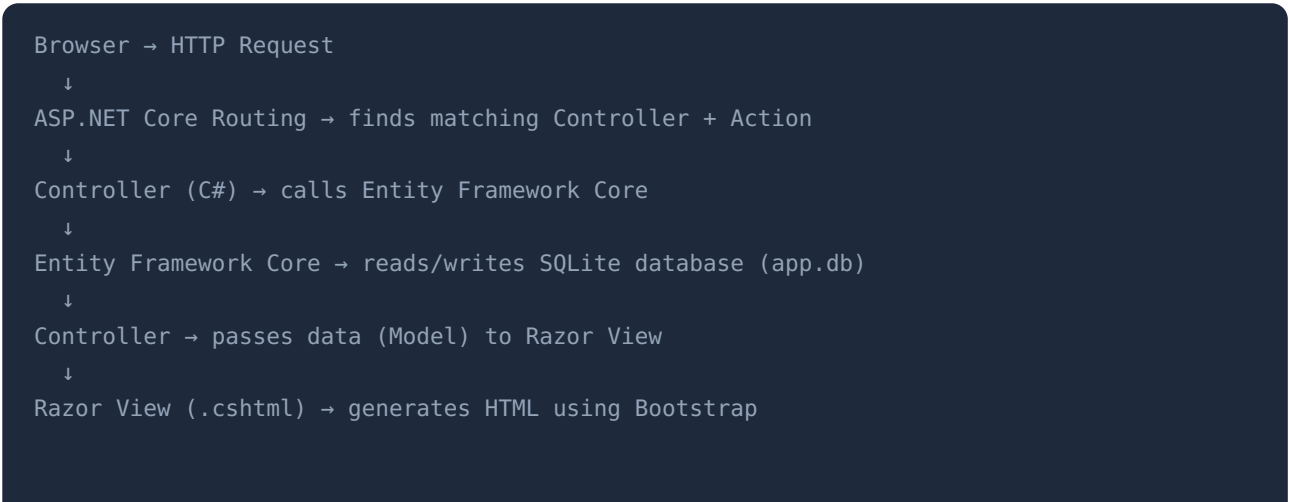
WHY TWO BRANCHES?

The adviser initially said not to include login. So `crud (copy)` was created as a stripped-down version without authentication. The full `crud` branch is the complete, production-ready system with all features and login.

2. Technology Stack

Technology	Version	What It Does in This System
ASP.NET Core MVC	8.0	The main web framework. Handles HTTP requests, routes URLs to controller actions, renders HTML views using Razor syntax (.cshtml files).
C#	12 (.NET 8)	The programming language used for all controllers, models, helpers, and business logic.
Entity Framework Core	8.0.0	ORM (Object-Relational Mapper) — lets us work with database tables as C# classes instead of writing raw SQL. Handles queries, inserts, updates, deletes.
SQLite	via EF Core	The database engine. Stores all data in a single file called <code>app.db</code> in the project root. Lightweight, no separate server needed.
Razor (.cshtml)	built-in	Template engine that mixes C# and HTML to generate dynamic web pages. Files live in the <code>Views/</code> folder.
Bootstrap 5	5.x (local)	CSS + JavaScript library for responsive UI. Buttons, tables, forms, navbar, badges, cards — all styled with Bootstrap classes. Stored locally in <code>wwwroot/lib/bootstrap/</code> .
Cookie Authentication	ASP.NET built-in	After login, a browser cookie is created containing the user's identity and role. Every page request checks this cookie to know who is logged in. Non-persistent — cookie dies when the browser closes.
PBKDF2-SHA256	built-in .NET	Password hashing algorithm. Passwords are never stored as plain text. 100,000 iterations, 16-byte random salt, 32-byte hash — very secure.
LibMan (Library Manager)	VS tool	Downloaded Bootstrap locally so the app works offline without CDN dependencies.

How the Technologies Connect





HTTP Response → Browser renders the page

3. Project Folder Structure

YnclinoApartmentManagementSystem/ | |— **Controllers/** ← C# classes that handle requests (the "C" in MVC) | |— AccountController.cs ← Login, Logout, Change Password | |— HomeController.cs ← Dashboard (admin view + tenant view) | |— UnitsController.cs ← CRUD for apartment units | |— TenantsController.cs ← CRUD for tenants | |— BillingController.cs ← CRUD for billing records | |— MaintenanceController.cs ← CRUD for maintenance requests | |— LostFoundController.cs ← CRUD for lost & found items + claims | |— UsersController.cs ← CRUD for staff accounts | |— **Models/** ← C# classes that represent database tables (the "M" in MVC) | |— tblUnit.cs ← Unit entity / table definition | |— tblTenant.cs ← Tenant entity | |— tblUser.cs ← User entity | |— tblBilling.cs ← Billing entity | |— tblMaintenanceRequest.cs ← Maintenance entity | |— tblLostFoundItem.cs ← Lost & Found entity | |— tblClaimRequest.cs ← Claim request entity (part of Lost & Found) | |— **ViewModels/** ← Form-specific models (validation rules, display logic) | |— UnitViewModel.cs | |— TenantViewModel.cs | |— UserViewModel.cs | |— BillingViewModel.cs | |— MaintenanceViewModel.cs | |— LostFoundViewModel.cs | |— LoginViewModel.cs | |— ChangePasswordViewModel.cs | |— **Views/** ← Razor HTML templates (the "V" in MVC) | |— **Account/** ← Login, ChangePassword, AccessDenied pages | |— **Home/** ← AdminDashboard, TenantDashboard pages | |— **Units/** ← Index, Create, Edit, Delete pages | |— **Tenants/** ← Index, Create, Edit, Details, Delete pages | |— **Billing/** ← Index, Create, Edit, Details pages | |— **Maintenance/** ← Index, Create, Edit, Details pages | |— **LostFound/** ← Index, Create, Details, Claim pages | |— **Users/** ← Index, Create, Edit, Delete pages | |— **Shared/** ← Layout & partials used by all pages | |— _Layout.cshtml ← Master page: navbar, footer, toasts | |— _ValidationScriptsPartial.cshtml | |— **Data/** | |— ApplicationDbContext.cs ← EF Core database context (all table mappings) | |— **Helpers/** | |— PasswordHelper.cs ← Hash() and Verify() for passwords (PBKDF2-SHA256) | |— **Migrations/** ← EF Core migration files (database schema history) | |— **wwwroot/** ← Static files served directly to the browser | |— **css/site.css** ← Custom CSS (password strength rules, pw-toggle) | |— **js/site.js** ← Custom JavaScript | |— **js/password.js** ← Password strength checker (checkStrength, meetsAllRules) | |— **lib/bootstrap/** ← Bootstrap 5 CSS + JS (stored locally) | |— **Properties/** | |— launchSettings.json ← Dev server port configuration | |— Program.cs ← App startup: services, middleware, seed data, routing | |— appsettings.json ← Config: database connection string | |— app.db ← The SQLite database file (auto-created on first run) | |— *.csproj ← Project file: NuGet package references

WHERE IS THE DATABASE?

`app.db` sits in the project root folder. You can open it with any SQLite browser (like DB Browser for SQLite) to view the raw data. It is automatically created when the app runs for the first time.

4. Database Design

The database has 7 tables on the full `crud` branch (3 on `crud(copy)`). All tables use integer primary keys named after the entity (e.g. `UnitID`, `TenantID`).

tblUsers — Staff & Tenant User Accounts

Column	Type	Notes
<code>UserID</code>	int (PK)	Auto-incremented primary key
<code>Username</code>	string (50)	Must be unique across all users
<code>Password</code>	string (255)	PBKDF2-SHA256 hash — never plain text
<code>Role</code>	string (20)	<code>Admin</code> <code>SemiAdmin</code> <code>Tenant</code>
<code>IsActive</code>	bool	Inactive users cannot log in
<code>IsSuperAdmin</code>	bool	Marks the initial admin — cannot be deleted or demoted
<code>DateCreated</code>	DateTime	Auto-set on record creation

tblUnits — Apartment Units

Column	Type	Notes
<code>UnitID</code>	int (PK)	Auto-incremented primary key
<code>UnitNumber</code>	string (20)	Unique — digits only (e.g. "101")
<code>UnitType</code>	string (50)	Studio 1-Bedroom 2-Bedroom 3-Bedroom
<code>RentPrice</code>	decimal (10,2)	Monthly rent in Philippine Peso
<code>Capacity</code>	int	Max number of tenants; range 1–100
<code>Status</code>	string (20)	Vacant Occupied Under Maintenance (default: Vacant)
<code>DateAdded</code>	DateTime	Auto-set when unit is created

tblTenants — Tenant Records

Column	Type	Notes
<code>TenantID</code>	int (PK)	Auto-incremented primary key
<code>UserID</code>	int? (FK → tblUsers)	Optional link to a login account; SET NULL on user delete
<code>UnitID</code>	int (FK → tblUnits)	Required — which unit is the tenant in; RESTRICT delete

FirstName	string (50)	Letters, spaces, hyphens, apostrophes only
LastName	string (50)	Same validation as FirstName
ContactNumber	string? (20)	Digits only, 7–15 characters
EmergencyContact	string? (100)	Digits only, 7–15 characters
MoveInDate	DateTime?	Date tenant physically moved in
MoveOutDate	DateTime?	Date tenant moved out (if applicable)
LeaseStart	DateTime?	Start of lease contract
LeaseEnd	DateTime?	Auto-set to 1 month after MoveInDate; still editable
Status	string (20)	Active Inactive (default: Active)
DateRecorded	DateTime	Auto-set when record is created

tblBillings — Billing & Payment Records

Column	Type	Notes
BillingID	int (PK)	Auto-incremented
TenantID	int (FK → tblTenants)	CASCADE on tenant delete
BillingPeriod	DateTime	Always the 1st day of the billing month
AmountDue	decimal (10,2)	Total amount to pay
DueDate	DateTime	Default: 30 days from issue date
AmountPaid	decimal? (10,2)	Null if not yet paid
DatePaid	DateTime?	Null if not yet paid
Status	string (20)	Unpaid Paid Overdue (default: Unpaid)
Notes	string? (500)	Optional admin notes
DateIssued	DateTime	Auto-set on creation

tblMaintenanceRequests — Maintenance Requests

Column	Type	Notes
RequestID	int (PK)	Auto-incremented
TenantID	int (FK → tblTenants)	CASCADE on tenant delete
Category	string (50)	Plumbing Electrical Structural Appliance Other

Description	string (500)	Required description of the problem
Priority	string (20)	Low Medium High (default: Medium)
Status	string (30)	Pending In Progress Resolved Cancelled (default: Pending)
DateSubmitted	DateTime	Auto-set on creation
DateResolved	DateTime?	Set when status changes to Resolved
AdminNotes	string? (500)	Notes added by Admin/SemiAdmin when updating status

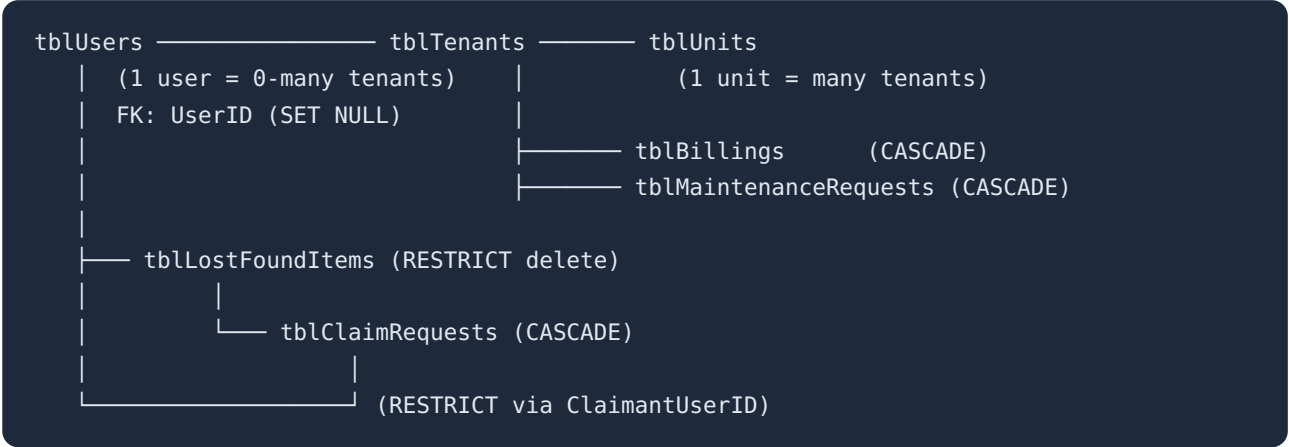
tblLostFoundItems — Lost & Found Items

Column	Type	Notes
ItemID	int (PK)	Auto-incremented
ReportedByUserID	int (FK → tblUsers)	RESTRICT on user delete
ItemName	string (100)	Required
Description	string? (500)	Optional detailed description
ItemType	string (10)	Lost Found
Location	string? (200)	Where it was lost or found
Status	string (20)	Reported Claimed Resolved (default: Reported)
DateReported	DateTime	Auto-set on creation
Notes	string? (500)	Additional notes

tblClaimRequests — Claim Requests (part of Lost & Found)

Column	Type	Notes
ClaimID	int (PK)	Auto-incremented
ItemID	int (FK → tblLostFoundItems)	CASCADE on item delete
ClaimantUserID	int (FK → tblUsers)	RESTRICT on user delete
VerificationDetails	string (1000)	Tenant's proof of ownership description
SubmittedAt	DateTime	Auto-set on submission
Status	string (20)	Pending Approved Rejected (default: Pending)
AdminNotes	string? (500)	Admin decision notes

Table Relationships Diagram



5. Authentication & Role-Based Access

How Login Works (Step by Step)

- 1 User visits any page → `[Authorize]` attribute redirects unauthenticated users to `/Account/Login`.
- 2 User enters username and password on the login form.
- 3 Server looks up username in `tblUsers` where `IsActive = true`.
- 4 `PasswordHelper.Verify()` re-hashes the typed password with the stored salt and compares byte-by-byte.
- 5 If matched, a **Claims Identity** is created (UserID, Username, Role) and stored in a browser cookie.
- 6 On every request, ASP.NET reads the cookie and populates `User.Identity` automatically.
- 7 Logout clears the cookie via `HttpContext.SignOutAsync()`.

The Three Roles

Role	Who They Are	Access Level
Admin	Building manager / owner. The first Admin is the Super Admin (marked with a star badge).	Full access to all modules. Can create/edit/delete staff accounts. Super Admin cannot be deleted.
SemiAdmin	Staff member (e.g. secretary, assistant).	Can view and manage all modules but cannot manage other staff accounts in most cases.
Tenant	A tenant with a login account linked to their tenant record.	Can see their own billing, submit maintenance requests, report/claim lost items. Cannot see other tenants' data.

Access Control Per Module

Module / Action	Admin	SemiAdmin	Tenant
View Units list	✓	✓	✓
Add / Edit / Delete Unit	✓	✓	✗
View all Tenants	✓	✓	✗
Register / Edit / Deactivate Tenant	✓	✓	✗
View all Billing records	✓	✓	Own only
Issue / Edit Billing	✓	✓	✗
View all Maintenance requests	✓	✓	Own only
Submit Maintenance request	✗	✗	✓

Update Maintenance status / notes	✓	✓	✗
View Lost & Found items	✓	✓	✓
Report Lost/Found item	✓	✓	✓
Claim a Found item	✗	✗	✓
Review / approve claims	✓	✓	✗
View User Management	✓	✓	✗
Create staff accounts	✓	✗	✗
Change own password	✓	✓	✓

HOW ACCESS CONTROL IS ENFORCED IN CODE

Controller classes are marked with `[Authorize]` (requires any login) or `[Authorize(Roles = "Admin,SemiAdmin")]` (requires specific role). Any user trying to access a restricted page is redirected to `/Account/AccessDenied`.

Default Admin Credentials

Username	Password	Role	IsSuperAdmin
admin	Admin@123	Admin	Yes

This account is automatically created on first run if no Admin exists (seeded in `Program.cs`).

Password Rules (enforced on Create/Change Password)

- At least 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character: `!@#$$%^&*`

6. Module: Units Management

What It Does

Manages all apartment units in the building. Each unit has a unique number, a type, a rent price, a capacity, and a status. Units are the foundation of the system — tenants must be assigned to a unit.

Where to Find It

Item	Location
URL	/Units
Controller	Controllers/UnitsController.cs
Views	Views/Units/ → Index, Create, Edit, Delete
Model	Models/tblUnit.cs
ViewModel	Models/ViewModels/UnitViewModel.cs
DB Table	tblUnits

Available Actions

Action	URL	Who Can	What It Does
List units	GET /Units	All logged-in	Shows table of all units with search and status filter
Add unit	GET/POST /Units/Create	Admin, SemiAdmin	Form to add a new unit; checks for duplicate unit numbers
Edit unit	GET/POST /Units/Edit/{id}	Admin, SemiAdmin	Edit any field of an existing unit
Delete unit	GET/POST /Units/Delete/{id}	Admin, SemiAdmin	Permanently deletes unit; blocked if active tenants exist

Special Logic

- **Unique unit number:** Duplicate unit numbers are rejected with a validation error.
- **Delete protection:** A unit cannot be deleted if it has active tenants (Status = "Active"). Admin must deactivate tenants first.
- **Auto price by type:** Selecting a unit type on the Create form auto-fills the price field: Studio ₱3,500 · 1BR ₱5,000 · 2BR ₱8,000 · 3BR ₱12,000. Price is still fully editable.
- **Price spinner:** The up/down arrows on the price field step by ₱100 at a time.
- **Search:** Filters by unit number or unit type (partial match).

- **Status filter:** Dropdown auto-submits on change (no Filter button).

Unit Fields (on Create/Edit form)

Field	Required?	Validation
Unit Number	Yes	Digits only; must be unique
Unit Type	Yes	One of: Studio, 1-Bedroom, 2-Bedroom, 3-Bedroom
Rent Price	Yes	₱0.01 – ₱999,999.99; steps of ₱100
Capacity	Yes	Integer 1–100
Status	Yes	Vacant Occupied Under Maintenance

7. Module: Tenants Management

What It Does

Manages tenant records — personal information, unit assignment, lease dates, and optionally a login account. Tenants are never permanently deleted; they are deactivated (soft delete). This preserves all historical data (billing records, maintenance requests, etc.).

Where to Find It

Item	Location
URL	/Tenants
Controller	Controllers/TenantsController.cs
Views	Views/Tenants/ → Index, Create, Edit, Details, Delete (deactivate)
Model	Models/tblTenant.cs
ViewModel	Models/ViewModels/TenantViewModel.cs
DB Table	tblTenants

Available Actions

Action	URL	Who Can	What It Does
List tenants	GET /Tenants	Admin, SemiAdmin	Shows table of tenants; defaults to Active filter; searchable
Register tenant	GET/POST /Tenants/ Create	Admin, SemiAdmin	Full form; optionally creates a linked login account
Edit tenant	GET/POST /Tenants/ Edit/{id}	Admin, SemiAdmin	Edit all details; can update/remove linked account
View details	GET /Tenants/Details/ {id}	All logged-in	Full read-only profile page; Tenant role can only see their own
Deactivate tenant	GET/POST /Tenants/ Delete/{id}	Admin, SemiAdmin	Sets Status = "Inactive"; record is NEVER deleted from DB

Tenant Fields (Create/Edit form)

Field	Required?	Validation
First Name	Yes	Letters, spaces, hyphens, apostrophes only (no numbers)

Last Name	Yes	Same as First Name
Unit	Yes	Dropdown of all units
Contact Number	No	Digits only, 7–15 characters
Emergency Contact	No	Digits only, 7–15 characters
Move-In Date	No	Date picker
Lease Start	No	Date picker
Lease End	No	Auto-fills to 1 month after Move-In; still editable
Move-Out Date	No	Date picker (optional)
Status	Yes	Active Inactive
Username	No	Only shown if creating a login account for this tenant
Password	No	Required only if Username is provided; must meet strength rules

Special Logic

- **Soft delete:** "Delete" actually sets `Status = "Inactive"`. The record, billing history, and maintenance history are all preserved.
- **Optional login:** When registering a tenant, you can optionally provide a Username + Password. This creates a `tblUser` record with `Role = "Tenant"` and links it to the tenant via `UserID`.
- **Auto-generated username:** The system can suggest a username from the tenant's name.
- **Lease End auto-fill:** Selecting a Move-In date automatically sets Lease End to the same date next month. The user can override this.
- **Cannot delete unit with active tenant:** The foreign key constraint uses `RESTRICT` — you cannot delete a unit that has tenants pointing to it.
- **Status filter default:** The Tenants list defaults to showing only Active tenants. Select "All" to see inactive ones.
- **Computed property:** `FullName` = `FirstName` + " " + `LastName` — not stored in DB, computed on the fly.

WHY SOFT DELETE?

Permanently deleting a tenant would also delete or orphan all their billing records, maintenance requests, and payment history. Soft delete keeps all that history intact while removing the tenant from active management.

8. Module: Billing & Payment

What It Does

Issues and tracks monthly rent bills for tenants. Admin/SemiAdmin can issue bills, record payments, and update status. Tenants can view their own billing history. The module also has a feature that suggests the amount due based on the tenant's unit's rent price.

Where to Find It

Item	Location
URL	<code>/Billing</code>
Controller	<code>Controllers/BillingController.cs</code>
Views	<code>Views/Billing/</code> → Index, Create, Edit, Details
Model	<code>Models/tblBilling.cs</code>
ViewModel	<code>Models/ViewModels/BillingViewModel.cs</code>
DB Table	<code>tblBillings</code>

Available Actions

Action	Who Can	What It Does
View billing list	Admin/SemiAdmin see all; Tenant sees own only	Table with search (by tenant name) and status filter
Issue bill	Admin, SemiAdmin	Create new billing record for a tenant; amount auto-suggested from unit's rent price
Edit billing	Admin, SemiAdmin	Update amount, status, payment details, notes
View details	All; tenants see own only	Full read-only view of a billing record

Billing Fields

Field	Notes
Tenant	Dropdown of active tenants; selecting one AJAX-fetches their unit's rent price
Billing Period	Defaults to the 1st of the current month
Amount Due	Suggested from tenant's unit rent; editable
Due Date	Defaults to 30 days from today

Amount Paid	Filled in when payment is received
Date Paid	Date payment was recorded
Status	Unpaid → Paid → Overdue (manually set; system does not auto-expire)
Notes	Optional remarks

Special Logic

- **AJAX amount suggestion:** The controller exposes a `GetSuggestedAmount` endpoint. When a tenant is selected on the Create form, JavaScript calls this endpoint to retrieve the unit's `RentPrice` and pre-fills the Amount Due field.
- **Tenant view:** When a tenant logs in, they only see their own billing records. Admin/SemiAdmin see everyone's.
- **No auto-status update:** The Overdue status is set manually by an admin — the system does not automatically mark bills as Overdue when the due date passes.
- **Cascade delete:** If a tenant record is ever hard-deleted (unusual), all their billing records are also deleted.

9. Module: Maintenance Requests

What It Does

Allows tenants to submit maintenance requests and lets admins track, prioritize, and resolve them. This is a two-role workflow: tenants create requests, admins manage them.

Where to Find It

Item	Location
URL	/Maintenance
Controller	Controllers/MaintenanceController.cs
Views	Views/Maintenance/ → Index, Create, Edit, Details
Model	Models/tblMaintenanceRequest.cs
ViewModel	Models/ViewModels/MaintenanceViewModel.cs
DB Table	tblMaintenanceRequests

Workflow

- 1 Tenant logs in and clicks "New Request" in the Maintenance module.
- 2 Tenant fills in Category, Description, Priority, and submits.
- 3 Request appears in admin's list with status "Pending".
- 4 Admin changes status to "In Progress" and optionally adds Admin Notes.
- 5 Admin marks as "Resolved" (Date Resolved is recorded) or "Cancelled".
- 6 Tenant can see the updated status and admin notes in their own view.

Request Fields

Field	Who Fills	Notes
Tenant	Auto (own) / Admin selects	Tenant role always sees only their own
Category	Tenant	Plumbing Electrical Structural Appliance Other
Description	Tenant	Required, up to 500 characters
Priority	Tenant	Low Medium High
Status	Admin / SemiAdmin	Pending → In Progress → Resolved Cancelled
Date Resolved	Admin / SemiAdmin	Set when marking Resolved

Admin Notes	Admin / SemiAdmin	Internal notes for communication
-------------	-------------------	----------------------------------

Special Logic

- **Role-based view filtering:** Tenants see only their own requests. Admin/SemiAdmin see all requests with a tenant name and unit column.
- **Priority badge colors:** High = red, Medium = yellow, Low = gray.
- **Status badge colors:** Resolved = green, In Progress = blue, Cancelled = gray, Pending = yellow.

10. Module: Lost & Found

What It Does

A community board for lost and found items within the apartment building. Anyone (admin or tenant) can report an item as Lost or Found. Tenants can submit a claim for a Found item, providing proof of ownership. Admins review and approve or reject claims.

Where to Find It

Item	Location
URL	<code>/LostFound</code>
Controller	<code>Controllers/LostFoundController.cs</code>
Views	<code>Views/LostFound/</code> → Index, Create, Details, Claim
Models	<code>Models/tblLostFoundItem.cs</code> , <code>Models/tblClaimRequest.cs</code>
DB Tables	<code>tblLostFoundItems</code> , <code>tblClaimRequests</code>

Item Lifecycle

- 1 Any user reports an item (Lost or Found) with name, location, and description.
- 2 Item appears in the list with status "Reported".
- 3 If item type is "Found" — a tenant can click "Claim" and submit proof of ownership.
- 4 Admin reviews the claim on the Details page and approves or rejects it.
- 5 If approved → item status changes to "Claimed". If resolved further → "Resolved".

Item Fields

Field	Notes
Item Name	Required, up to 100 characters
Type	Lost Found
Location	Where it was lost or found (optional)
Description	Additional details (optional, up to 500 chars)
Status	Reported Claimed Resolved
Notes	Additional remarks (optional)
Reported By	Auto-set to logged-in user

Claim Request Fields

Field	Notes
Verification Details	Required — tenant describes proof of ownership (up to 1000 chars)
Status	Pending Approved Rejected
Admin Notes	Admin's decision reasoning

Filter Options

The Lost & Found index has **two dropdowns**: one filters by type (Lost/Found), the other by status (All/Reported/Claimed/Resolved). Both auto-submit on change.

11. Module: User Management

What It Does

Manages staff login accounts (Admin and SemiAdmin roles). This is separate from tenant accounts — tenant accounts are created inside the Tenants module. The User Management page is only visible to Admin and SemiAdmin roles in the navigation bar.

Where to Find It

Item	Location
URL	/Users
Nav label	User Management
Controller	Controllers/UsersController.cs
Views	Views/Users/ → Index, Create, Edit, Delete
Model	Models/tblUser.cs
ViewModel	Models/ViewModels/UserViewModel.cs
DB Table	tblUsers

Available Actions

Action	Who Can	What It Does
View list	Admin, SemiAdmin	Shows all accounts with role/status badges
Add staff account	Admin only	Creates SemiAdmin or Admin account with password strength check
Edit account	Admin (all); SemiAdmin (own Tenant accounts only)	Edit username, role, active status; only Super Admin can reset passwords
Delete account	Admin only; Super Admin is undeletable	Hard delete; linked tenant's UserID is set to NULL (tenant record preserved)

Special Logic

- **Super Admin protection:** The original admin (IsSuperAdmin = true) cannot be deleted and cannot have their status set to Inactive from the edit form.
- **Password reset:** Only the Super Admin can set another account's password from the Edit form.
- **Linked tenants preserved:** When a staff account is deleted, any tenants who had that account linked just lose the link (UserID = NULL). Their tenant records are not deleted.

- **Star badge:** The Super Admin account shows a badge next to their username in the list.
- **Tenant accounts not shown here:** The Users index only shows Admin and SemiAdmin accounts. Tenant accounts are managed from the Tenants module.

12. Dashboard (Home)

The dashboard (`/` or `/Home`) shows different content depending on the user's role.

Admin / SemiAdmin Dashboard

Shows live summary statistics pulled from the database:

- Total Units
- Vacant Units
- Occupied Units
- Units Under Maintenance
- Active Tenants
- Inactive Tenants
- Total Active Users

View file: `Views/Home/AdminDashboard.cshtml`

Tenant Dashboard

Shows the logged-in tenant's own profile summary:

- Their name and unit number
- Lease dates
- Contact info
- Quick links to Billing and Maintenance

View file: `Views/Home/TenantDashboard.cshtml`

HOW THE CONTROLLER DECIDES WHICH VIEW TO SHOW

`HomeController.Index()` checks `User.IsInRole("Admin") || User.IsInRole("SemiAdmin")` and returns either `AdminDashboard` or `TenantDashboard` accordingly.

13. MVC Pattern — How It Applies Here

MVC stands for **Model–View–Controller**. It separates the application into three layers, each with a distinct job.

Layer	Folder in Project	Job	Example
Model	Models/	Defines the data structure and business rules. Maps to database tables. Holds validation annotations.	<code>tblUnit.cs</code> defines that <code>UnitNumber</code> must be required and max 20 chars.
View	Views/	The HTML page the user sees. Uses Razor syntax to show data passed from the controller. Never queries the database directly.	<code>Views/Units/Index.cshtml</code> loops through a list of units and renders a table row for each.
Controller	Controllers/	The traffic director. Receives HTTP requests, reads/writes data via EF Core, then passes the result to a View or redirects.	<code>UnitsController.Index()</code> queries the database for units, filters them, and passes the list to the View.

Concrete Request Flow Example: Adding a Unit

- 1 Browser** → User clicks "+ Add Unit". Browser sends GET request to `/Units/Create`.
- 2 Router** → ASP.NET matches URL to `UnitsController.Create()` (GET method).
- 3 Controller** → Creates an empty `UnitViewModel` and returns `View(vm)`.
- 4 View** → `Views/Units/Create.cshtml` renders the blank form with Bootstrap styling.
- 5 Browser** → User fills form and clicks "Save Unit". Browser sends POST to `/Units/Create`.
- 6 Controller** → `Create(UnitViewModel vm)` is called. Model state is validated. Duplicate check runs. A `tblUnit` entity is created and saved via EF Core.
- 7 Controller** → Sets a `TempData["Success"]` toast message and redirects to `/Units`.
- 8 Browser** → Units list page loads. Toast notification appears at bottom right.

ViewModels vs Models — Why Both?

Database models (`tblUnit`) mirror the database exactly. ViewModels (`UnitViewModel`) are tailored for forms — they can have extra validation rules, dropdown lists, computed display properties, and fields not in the database. This keeps the database model clean and the form logic separate.

14. Key Files & What They Do

File	What It Does
<code>Program.cs</code>	App entry point. Registers all services (EF Core, authentication, MVC). Configures middleware pipeline (routing, auth, static files). Seeds the default admin user. On <code>crud(copy)</code> , contains middleware that auto-logs in as admin for demo purposes.
<code>appsettings.json</code>	Contains the database connection string: <code>"DefaultConnection": "Data Source=app.db"</code> . This tells EF Core to use SQLite and where the file is.
<code>Data/ ApplicationDbContext.cs</code>	The EF Core "bridge" between C# classes and the database. Declares all 7 <code>DbSet</code> tables and configures constraints (unique indexes, foreign keys, delete behaviors) in <code>OnModelCreating()</code> .
<code>Helpers/ PasswordHelper.cs</code>	Two static methods: <code>Hash(password)</code> — generates a secure hash for storing. <code>Verify(password, storedHash)</code> — checks a typed password against the stored hash. Uses PBKDF2-SHA256 with 100,000 iterations.
<code>Views/Shared/ _Layout.cshtml</code>	The master page template. All other pages insert into it via <code>@RenderBody()</code> . Contains the navbar (with role-based visibility), footer, Bootstrap JS, and the toast notification system.
<code>wwwroot/js/password.js</code>	Client-side password strength checker. Exports <code>checkStrength()</code> (updates rule indicators in real time) and <code>meetsAllRules()</code> (returns true/false for form submission validation).
<code>wwwroot/css/site.css</code>	Custom styles — password strength rule list, show/hide password button (<code>.pw-toggle</code> , <code>.pw-wrapper</code>), shake animation for failed password rules.
<code>Migrations/</code>	EF Core migration files — each file represents a database schema change. The <code>InitialCreate</code> migration creates all tables. Run via <code>db.Database.Migrate()</code> in <code>Program.cs</code> on startup.
<code>Properties/ launchSettings.json</code>	Development server configuration. <code>crud</code> branch uses port 7251. <code>crud(copy)</code> uses port 7250. This allows both to run simultaneously in development.

15. How to Demo the System

BEFORE STARTING

Make sure the project is running. In Visual Studio, press F5 or click the green Run button. The browser should open at `https://localhost:7251` (crud) or `https://localhost:7250` (crud(copy)).

Recommended Demo Order (Full crud branch)

Step 1 — Login

- 1 Go to `/Account/Login`. Enter username `admin` and password `Admin@123`. Click Sign In.
- 2 You land on the Admin Dashboard showing unit and tenant counts.

Step 2 — Add a Unit

- 1 Click "Units" in the navbar → click "+ Add Unit".
- 2 Enter Unit Number (e.g. 101), select "Studio" — watch the price auto-fill to ₱3,500.
- 3 Set capacity to 2, status Vacant. Click "Save Unit". Toast: "Unit 101 has been added."

Step 3 — Register a Tenant

- 1 Click "Tenants" → "+ Register Tenant".
- 2 Enter First Name, Last Name. Select Unit 101. Add a Move-In Date — Lease End auto-fills.
- 3 Optionally enter a Username/Password to give the tenant a login account.
- 4 Click Save. The tenant appears in the list.

Step 4 — Issue a Bill

- 1 Click "Billing & Payment" → "+ Issue Bill".
- 2 Select the tenant — Amount Due auto-fills from their unit's rent price.
- 3 Set billing period and due date. Click Save.

Step 5 — Submit a Maintenance Request (as Tenant)

- 1 Log out. Log in as the tenant account you created.
- 2 Click "Maintenance" → "+ New Request". Fill in Category, Description, Priority.
- 3 Log back in as admin. See the new request in the Maintenance list.
- 4 Edit it → change status to "In Progress", add Admin Notes, click Save.

Step 6 — Report a Lost Item

- 1 Click "Lost & Found" → "+ Report Item". Select type "Lost", fill in item name and location.
- 2 The item appears in the list with status "Reported".

Step 7 — Add a Staff Account

- 1 Click "User Management" → "+ Add Staff Account".
- 2 Enter username, set a strong password, choose role SemiAdmin. Click Create.

16. Glossary of Technical Terms

Term	Plain English Explanation
ASP.NET Core MVC	Microsoft's framework for building web applications in C#. "Core" means it runs on Linux/Mac/Windows. "MVC" is the design pattern it follows (Model-View-Controller).
Entity Framework Core (EF Core)	A tool that lets you work with a database using C# objects (classes) instead of writing SQL commands manually. It automatically translates C# operations into SQL.
SQLite	A lightweight database stored as a single file (app.db). No separate database server is needed — the database runs inside the application itself. Good for development and small-scale apps.
ORM	Object-Relational Mapper — a tool (like EF Core) that maps database tables to programming objects so you don't have to write SQL manually.
Migration	A file that describes a database schema change (e.g., add a table, add a column). EF Core applies migrations to keep the database structure in sync with your C# models.
Razor (.cshtml)	A template syntax that lets you mix C# code with HTML. The @ symbol starts a C# expression. Files end in .cshtml and are compiled into HTML on the server before being sent to the browser.
Bootstrap 5	A popular CSS framework that provides ready-made styles and components (buttons, tables, forms, navigation bars). Used here offline (files stored locally, not loaded from the internet).
ViewModel	A C# class designed specifically for a form or view. It can have validation rules, dropdown lists, and display-only properties that aren't stored in the database.
Controller Action	A method inside a Controller class that handles one HTTP request. E.g., <code>UnitsController.Create()</code> handles both GET (show the form) and POST (save the form).
Cookie Authentication	After logging in, the server creates a small encrypted file (cookie) stored in the browser. Every page request sends this cookie back to the server to identify the user without requiring a new login.
Claims	Pieces of identity information stored in the login cookie: UserID, Username, Role. Controllers read these via <code>User.FindFirstValue()</code> or <code>User.IsInRole()</code> .
[Authorize] attribute	A tag placed on a controller or method that tells ASP.NET "only let logged-in users access this." <code>[Authorize(Roles="Admin")]</code> further restricts to a specific role.
PBKDF2-SHA256	A password hashing algorithm. It scrambles a password using a random salt and 100,000 repetitions of SHA-256. The result cannot be reversed to get the original password.
Salt	A random 16-byte value added to a password before hashing. Two users with the same password will have different hashes because their salts differ.

Foreign Key (FK)	A column that stores the ID of a record in another table, creating a relationship. Example: <code>tblTenants.UnitID</code> links each tenant to their unit in <code>tblUnits</code> .
Cascade Delete	When a parent record is deleted, all related child records are automatically deleted too. Example: deleting a tenant also deletes all their billing records.
SET NULL on delete	When a parent record is deleted, the foreign key in related records is set to NULL instead of deleting the child. Example: deleting a user account sets <code>tblTenants.UserID = NULL</code> rather than deleting the tenant.
RESTRICT delete	Prevents a parent record from being deleted if child records reference it. Example: you cannot delete a unit if active tenants are assigned to it.
Soft Delete	Instead of removing a record from the database, it is marked as inactive (e.g., <code>Status = "Inactive"</code>). The record is hidden from normal views but preserved for history.
Hard Delete	Permanently removing a record from the database. Cannot be undone. Used for units and staff accounts.
TempData	A short-lived storage in ASP.NET that survives exactly one redirect. Used here to pass success/error messages to the next page, where they appear as toast notifications.
Antiforgery Token	A hidden security value embedded in every HTML form. The server checks it on POST requests to ensure the form was submitted from a legitimate page, not from a cross-site attack (CSRF protection).
DbContext	The EF Core class (<code>ApplicationDbContext</code>) that represents a session with the database. Used in controllers to query and save data: <code>_context.tblUnits.ToListAsync()</code> .
LINQ	Language-Integrated Query — C# syntax for querying collections and databases. Example: <code>.Where(u => u.Status == "Vacant")</code> filters a list of units.
Namespace	A grouping for C# classes to avoid name conflicts. All classes in this system use the namespace <code>YnclinoApartmentManagementSystem</code> .

17. Adviser Q & A

Q: What is the purpose of this system?

The Ynclino Apartment Management System is a web-based system for managing an apartment building. It handles unit management, tenant registration, billing and payments, maintenance requests, and a lost-and-found board. It also provides user account management with role-based access so admins, staff, and tenants each see only what is relevant to them.

Q: What technology/framework did you use and why?

We used ASP.NET Core 8.0 MVC with C# because it is a well-established, Microsoft-backed framework that enforces the MVC design pattern. SQLite was chosen as the database for its simplicity — it requires no separate database server installation, making the system easy to run on any development machine. Entity Framework Core handles all database operations so we don't write raw SQL.

Q: Explain the MVC pattern as used in your system.

Model: C# classes in the Models folder represent database tables and carry validation rules (e.g., `tblUnit.cs`). View: Razor `.cshtml` files in the Views folder are the HTML pages users see. Controller: C# classes in Controllers handle HTTP requests — they query the database using EF Core and pass data to views. For example, when a user opens `/Units`, the `UnitsController` queries all unit records, then passes the list to `Views/Units/Index.cshtml` to render.

Q: How many database tables does the system have?

Seven tables: `tblUsers`, `tblUnits`, `tblTenants`, `tblBillings`, `tblMaintenanceRequests`, `tblLostFoundItems`, and `tblClaimRequests`. The full crud branch uses all seven. The demo crud(copy) branch uses only `tblUsers`, `tblUnits`, and `tblTenants`.

Q: How does authentication work?

The system uses cookie-based authentication built into ASP.NET. When a user logs in, their username and role are stored in an encrypted browser cookie. Each page request checks this cookie. Passwords are never stored in plain text — they are hashed using PBKDF2-SHA256 with a random salt and 100,000 hashing iterations. The `[Authorize]` attribute on controllers prevents unauthenticated access.

Q: What are the user roles and what can each do?

Admin: full access to everything, including managing staff accounts. SemiAdmin: can manage units, tenants, billing, maintenance, and lost & found, but cannot create or delete staff accounts. Tenant: can view their own billing, submit maintenance requests, and report or claim lost items. They cannot see other tenants' data.

Q: Why is a tenant never deleted — only deactivated?

Permanently deleting a tenant would cascade-delete or orphan all their associated records — billing history, payment records, maintenance requests. We use a soft delete pattern: setting Status = "Inactive" hides the tenant from active views but preserves all historical data. This is important for record-keeping and auditing.

Q: What happens when you try to delete a unit that has tenants?

The system blocks it. The delete confirmation page checks if any active tenants are assigned to that unit. If yes, it redirects back with an error message: "Cannot delete a unit with active tenants." The foreign key relationship uses RESTRICT behavior, enforcing this at the database level as well.

Q: How is the database created and structured?

The database is a SQLite file called app.db, automatically created in the project root when the application runs for the first time. Entity Framework Core Migrations define the schema. On startup (Program.cs), `db.Database.Migrate()` applies any pending migrations. The `ApplicationDbContext.cs` file defines all table mappings and relationships (foreign keys, unique constraints, default values).

Q: What are the two branches and why are they different?

The crud branch is the complete production-ready system with all 6 modules, login, and role-based access. The crud(copy) branch is a stripped-down demo version with only Units and Tenants management and no login requirement — it was created when our adviser asked us to not include login yet. The two branches run on different ports (7251 and 7250) so both can run simultaneously.

Q: How does the Billing module know the rent amount for a tenant?

When issuing a bill, selecting a tenant triggers a JavaScript call to a `BillingController.GetSuggestedAmount()` endpoint. This endpoint looks up the tenant's assigned unit and returns that unit's `RentPrice`. The amount field is then pre-filled with that value. The admin can still change it manually if needed.

Q: How does the Lease End auto-fill work?

In the Tenant Create/Edit form, a JavaScript event listener watches the Move-In Date field. When a date is selected, the script calculates a date exactly one month later and sets it as the Lease End date. The Lease End field remains editable so the admin can override it if the actual lease term is different.

Q: How are passwords protected?

The PasswordHelper.cs class uses PBKDF2 with SHA-256, 100,000 iterations, and a 16-byte random salt. The salt ensures that two users with the same password will have different stored hashes. The 100,000 iterations make brute-force attacks computationally expensive. The hash is stored as a base64 string (salt + hash combined). Verification uses FixedTimeEquals to prevent timing attacks.

Q: What validation does the system perform?

Server-side validation uses Data Annotations (e.g., [Required], [MaxLength], [Range], [RegularExpression]) on ViewModel classes. EF Core also enforces constraints at the database level (unique indexes, NOT NULL). Client-side validation uses jQuery Unobtrusive Validation (automatically wired by ASP.NET). Custom JavaScript in password.js handles real-time password strength feedback with visual indicators for each rule.